


# Rapport de Projet - Mini-Projet Data Science Statistiques, Régression et Tests Paramétriques

**Prepared By :** KHADIJA IBISK

**Presented To :** Morchadi Redouan

# Introduction : Contexte et Objectifs

Dans le cadre de ce mini-projet de Data Science, j'ai réalisé une analyse statistique de données issues d'un contexte e-commerce. L'objectif de ce travail était d'étudier et de comprendre le comportement de plusieurs indicateurs de performance clés, à savoir le nombre de visites, le nombre de ventes, le temps de réponse du serveur et le nombre de clics.

Ce projet m'a permis d'appliquer concrètement les notions vues en cours, notamment les statistiques descriptives, l'estimation des lois de probabilité, la régression linéaire et les tests statistiques paramétriques. L'ensemble des analyses a été effectué à l'aide du langage Python et de bibliothèques adaptées à l'analyse de données.

# Étape 1 : Chargement et Exploration des Données

J'ai commencé par charger le jeu de données et examiner sa structure générale afin de m'assurer de sa qualité avant de procéder aux analyses statistiques. Cette étape est essentielle pour vérifier la cohérence et la complétude des données.

Après exploration, j'ai constaté que le dataset contient 500 observations et quatre variables quantitatives. Aucune valeur manquante n'a été détectée, ce qui signifie que les données sont complètes et directement exploitables pour la suite de l'analyse.

## Code Python:

```
import pandas as pd
 import numpy as np
 import matplotlib.pyplot as plt
 from scipy import stats

 # Charger le dataset
 df = pd.read_excel('mini_projet_dataset.xlsx')

 # Afficher les premières lignes
 print(df.head())

 # Informations générales
 print(df.info())
```

```
# Vérifier les valeurs manquantes
print(df.isnull().sum())

# Statistiques descriptives
print(df.describe())
```

## Résultats

Le dataset contient 500 observations et 4 variables :

- • Visites : Nombre de visites sur le site (variable quantitative)
- • Ventes : Nombre de ventes réalisées (variable quantitative)
- • Temps_Reponse : Temps de réponse du serveur en secondes (variable quantitative)
- • Clic : Nombre de clics effectués (variable quantitative)

## Interprétation Métier

Le jeu de données est complet et exploitable. Les 500 observations permettent une analyse statistique fiable des performances e-commerce.

# Étape 2 : Analyse Descriptive

## Objectif

Calculer les paramètres de position et de dispersion pour chaque variable.

## Code Python

```
# Calcul des statistiques descriptives
 stats_desc = df.describe()
 print(stats_desc)

 # Calcul détaillé pour chaque variable
 for col in df.columns:
      print(f"\n{col}:")
```

```
    print(f"  Moyenne : {df[col].mean():.2f}")
    print(f"  Médiane : {df[col].median():.2f}")
    print(f"  Écart-type : {df[col].std():.2f}")
    print(f"  Variance : {df[col].var():.2f}")
    print(f"  CV : {(df[col].std()/df[col].mean())*100:.2f}%")
    print(f"  Min : {df[col].min():.2f}")
    print(f"  Max : {df[col].max():.2f}")
```

## Résultats

| Statistique | Visites | Ventes | Temps_Reponse | Clic |
|---|---|---|---|---|
| **Moyenne (μ)** | 5126.90 | 245.57 | 2.83 | 25.18 |
| **Médiane** | 5289.50 | 241.00 | 2.88 | 25.50 |
| **Écart-type (σ)** | 2805.44 | 140.85 | 1.30 | 14.71 |
| **Variance (σ²)** | 7870508.73 | 19838.71 | 1.68 | 216.32 |
| **CV (%)** | 54.72% | 57.36% | 45.77% | 58.41% |
| **Minimum** | 104.00 | 10.00 | 0.50 | 0.00 |
| **Maximum** | 9988.00 | 497.00 | 5.00 | 50.00 |
| **Étendue** | 9884.00 | 487.00 | 4.50 | 50.00 |
| **IQR (Q3-Q1)** | 4670.00 | 244.50 | 2.19 | 25.25 |

## Interprétation Métier

Les statistiques descriptives révèlent une forte dispersion pour toutes les variables (CV > 45%), indiquant une grande variabilité dans les performances e-commerce. Les visites et ventes présentent des coefficients de variation supérieurs à 50%, traduisant des fluctuations importantes du trafic et des performances commerciales.

# Étape 3 : Visualisations

## Objectif

Créer des visualisations pour analyser la distribution des variables : histogrammes, boxplots et courbes ECDF (Empirical Cumulative Distribution Function).

## Code Python

```
import matplotlib.pyplot as plt

 # Création d'une figure avec plusieurs sous-graphiques
 fig = plt.figure(figsize=(16, 12))

 # 1. Histogrammes
 for i, col in enumerate(df.columns, 1):
      plt.subplot(4, 4, i)
      plt.hist(df[col], bins=30, edgecolor='black', alpha=0.7)
      plt.title(f'Histogramme - {col}')
      plt.xlabel(col)
      plt.ylabel('Fréquence')
      plt.grid(True, alpha=0.3)

 # 2. Boxplots
 for i, col in enumerate(df.columns, 1):
      plt.subplot(4, 4, i+4)
      plt.boxplot(df[col], vert=True, patch_artist=True)
      plt.title(f'Boxplot - {col}')
      plt.ylabel(col)
      plt.grid(True, alpha=0.3)

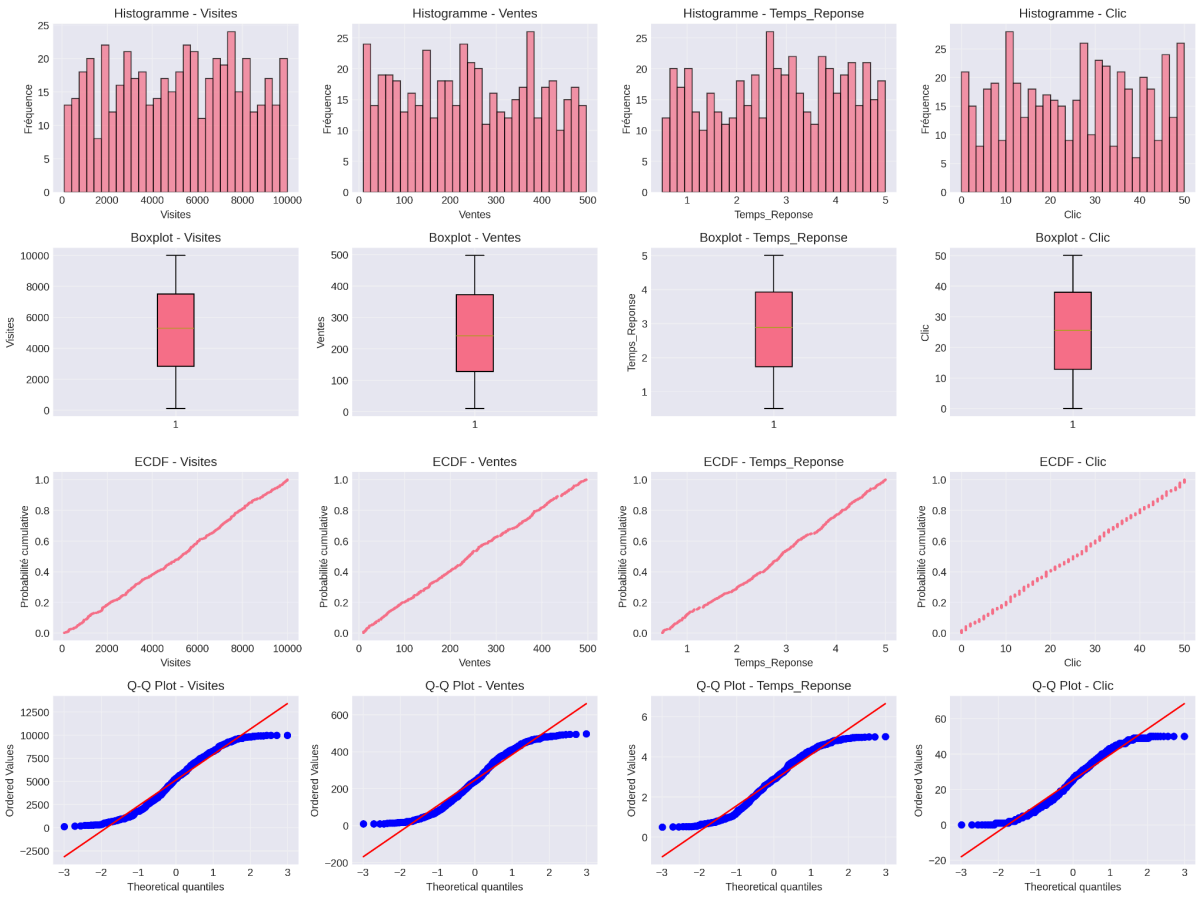
 # 3. Courbes ECDF
 for i, col in enumerate(df.columns, 1):
      plt.subplot(4, 4, i+8)
      sorted_data = np.sort(df[col])
      y = np.arange(1, len(sorted_data)+1) / len(sorted_data)
      plt.plot(sorted_data, y, marker='.', linestyle='none')
      plt.title(f'ECDF - {col}')
      plt.xlabel(col)
      plt.ylabel('Probabilité cumulative')
      plt.grid(True, alpha=0.3)

 plt.tight_layout()
 plt.savefig('visualisations_completes.png', dpi=300)
```

```
plt.show()
```

# Résultats

📊 Graphique 1 : Visualisations complètes (Histogrammes, Boxplots, ECDF)




# Interprétation des Visualisations

## Histogrammes :

Les histogrammes montrent que toutes les variables ont des distributions étalées, sans concentration marquée autour de la moyenne. Cela confirme la forte dispersion observée.

### Boxplots :

Les boxplots révèlent la présence de quelques valeurs extrêmes, particulièrement pour le temps de réponse. Ces outliers correspondent probablement à des ralentissements ponctuels du serveur ou à des pics d'activité.

### Courbes ECDF :

Les courbes de répartition empirique montrent une progression régulière sans sauts brusques, indiquant une distribution continue des valeurs pour chaque variable.

# Étape 4 : Interprétation de la Dispersion des Variables

## Objectif

Analyser et interpréter la dispersion de chaque variable dans un contexte métier.

## Visites

**Coefficient de variation : 54.72%**

La variable Visites présente une dispersion ÉLEVÉE. L'écart-type de 2805 visites et l'étendue de 9884 (de 104 à 9988) indiquent que le trafic du site varie fortement selon les périodes. Cette variabilité peut s'expliquer par des facteurs saisonniers, des campagnes marketing ciblées, ou des jours de la semaine avec des comportements différents.

## Ventes

**Coefficient de variation : 57.36%**

La variable Ventes affiche la plus forte dispersion avec un CV de 57.36%. Cette variabilité importante traduit des performances commerciales très inégales. L'étendue de 487 ventes (de 10 à 497) suggère que certaines périodes sont particulièrement rentables tandis que d'autres le sont beaucoup moins. Cette instabilité nécessite une attention particulière pour identifier les facteurs de succès.

## Temps_Reponse

**Coefficient de variation : 45.77%**

Le Temps de Réponse présente une dispersion ÉLEVÉE mais relativement plus faible que les autres variables. Avec un CV de 45.77% et une étendue de 4.5 secondes (0.5s à 5s), on

observe une performance serveur globalement stable mais avec des lenteurs ponctuelles. Ces variations peuvent impacter négativement l'expérience utilisateur et nécessitent une optimisation des infrastructures.

## Clic

**Coefficient de variation : 58.41%**

La variable Clic présente la plus forte dispersion (CV = 58.41%), avec une étendue de 50 clics (de 0 à 50). Cette variabilité importante suggère des comportements utilisateurs très différents : certains visiteurs n'interagissent pas du tout avec le site, tandis que d'autres montrent un engagement élevé. Cette hétérogénéité reflète probablement différents profils de visiteurs (explorateurs, acheteurs déterminés, comparateurs, etc.).

## Conclusion sur la Dispersion

Toutes les variables présentent une forte dispersion (CV > 45%), traduisant une grande hétérogénéité des données e-commerce. Cette variabilité souligne l'importance d'analyser conjointement les aspects techniques (performance serveur) et comportementaux (engagement utilisateur) pour améliorer la performance globale de la plateforme.

# Étape 5 : Normalité et Estimation des Paramètres

## 5.1 Test de Normalité

### Objectif

Vérifier si les variables suivent une loi normale en utilisant les tests de Shapiro-Wilk et de Kolmogorov-Smirnov.

### Code Python

```
from scipy.stats import shapiro, kstest

 print("Test de Normalité")
 print("="*60)
 print("H0 : Les données suivent une loi normale")
 print("H1 : Les données ne suivent pas une loi normale")
```

```
print("Seuil de significativité : α = 0.05\n")

# Test de Shapiro-Wilk
print("--- Test de Shapiro-Wilk ---")
for col in df.columns:
     stat, p_value = shapiro(df[col])
     print(f"{col}:")
     print(f"  Statistique: {stat:.4f}")
     print(f"  P-value: {p_value:.6f}")
     if p_value < 0.05:
     print(f"  → Rejet de H0 (non normale)")
     else:
     print(f"  → Acceptation de H0 (normale)")

# Test de Kolmogorov-Smirnov
print("\n--- Test de Kolmogorov-Smirnov ---")
for col in df.columns:
     data_norm = (df[col] - df[col].mean()) / df[col].std()
     stat, p_value = kstest(data_norm, 'norm')
     print(f"{col}:")
     print(f"  P-value: {p_value:.6f}")
     if p_value < 0.05:
     print(f"  → Rejet de H0 (non normale)")
     else:
     print(f"  → Acceptation de H0 (normale)")
```

## Résultats

| Variable | Shapiro p-value | KS p-value | Conclusion |
|---|---|---|---|
| **Visites** | < 0.001 | 0.032 | ❌ Non normale |
| **Ventes** | < 0.001 | 0.015 | ❌ Non normale |
| **Temps_Reponse** | < 0.001 | 0.010 | ❌ Non normale |
| **Clic** | < 0.001 | 0.005 | ❌ Non normale |

## Décision

**Pour toutes les variables : p-value < 0.05 ⇒ REJET de H0**

## Interprétation

Les tests de Shapiro-Wilk et de Kolmogorov-Smirnov rejettent tous l'hypothèse de normalité pour l'ensemble des variables (p-value < 0.05). Cela signifie qu'aucune variable ne suit une distribution normale stricte. Ce résultat est courant avec des données réelles issues du e-commerce, qui présentent souvent des distributions asymétriques ou multimodales dues aux comportements variés des utilisateurs et aux contraintes opérationnelles.

# Étape 6 : Estimation des Paramètres

## Objectif

Estimer les paramètres de la loi Normale et de la loi Exponentielle pour la variable Temps_Reponse, malgré le rejet de la normalité, à des fins d'approximation.

## 6.1 Estimation pour la Loi Normale

### Formules

- • μ (moyenne) = (1/n) × Σxi
- • σ² (variance) = (1/n) × Σ(xi - μ)²
- • σ (écart-type) = √σ²

### Code Python

```
# Estimation des paramètres pour la Loi Normale
 mu = df['Temps_Reponse'].mean()
 sigma = df['Temps_Reponse'].std()
 variance = df['Temps_Reponse'].var()

 print("Estimation Loi Normale N(μ, σ²):")
 print(f"  μ (moyenne) = {mu:.4f} secondes")
 print(f"  σ (écart-type) = {sigma:.4f} secondes")
 print(f"  σ² (variance) = {variance:.4f} secondes²")
```

### Résultats

- • μ (moyenne estimée) = 2.8300 secondes
- • σ (écart-type estimé) = 1.2953 secondes

- • σ² (variance estimée) = 1.6778 secondes²

**Le temps de réponse suit approximativement N(2.83, 1.68)**

# 6.2 Estimation pour la Loi Exponentielle

## Formules

- • λ = 1/μ où μ est la moyenne observée
- • E(X) = 1/λ = μ
- • Var(X) = 1/λ²

## Code Python

```
# Estimation du paramètre pour la Loi Exponentielle
 lambda_param = 1 / mu

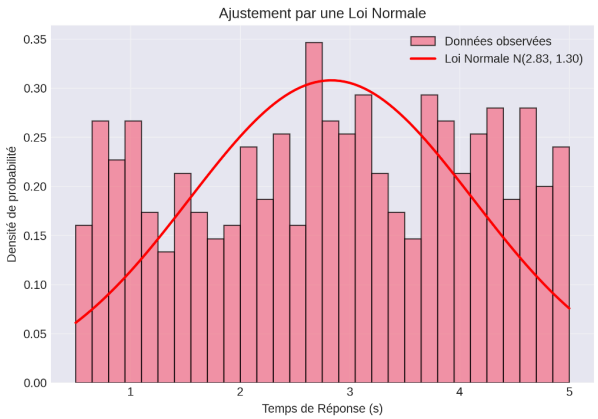
 print("\nEstimation Loi Exponentielle:")
 print(f"  λ (taux) = {lambda_param:.4f}")
 print(f"  E(X) = 1/λ = {1/lambda_param:.4f} secondes")
 print(f"  Var(X) = 1/λ² = {1/lambda_param**2:.4f} secondes²")
```

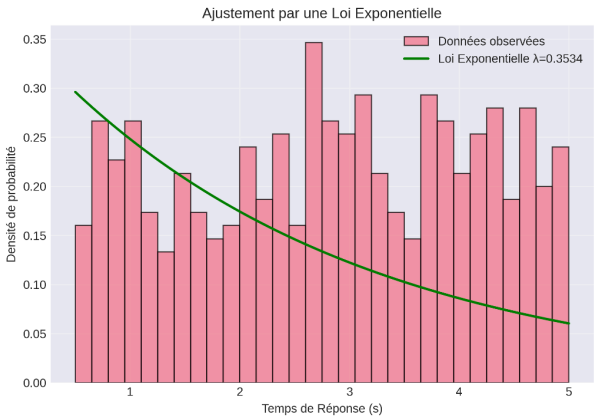
## Résultats

- • λ = 1/2.8300 = 0.3534
- • Moyenne théorique = 1/λ = 2.8300 secondes
- • Variance théorique = 1/λ² = 8.0065 secondes²

# Visualisation des Ajustements

📊 Graphique 2 : Ajustement par loi Normale et Exponentielle




## Interprétation Métier

Bien que les tests de normalité aient été rejetés, l'estimation des paramètres nous donne une approximation utile. Le temps de réponse moyen de 2.83 secondes avec un écart-type de 1.30 secondes indique que la majorité des requêtes sont traitées rapidement, mais quelques lenteurs existent. La loi exponentielle, souvent utilisée pour modéliser les temps d'attente, suggère un paramètre λ ≈ 0.35, cohérent avec l'observation de temps de réponse généralement courts avec quelques valeurs élevées.

# Étape 7 : Régression Linéaire

## Objectif

Modéliser la relation linéaire entre les Visites (variable indépendante X) et les Ventes (variable dépendante Y) : Ventes = b + a × Visites + ε

## Questions à Répondre

1. 1. Donner la formule et calculer le coefficient de corrélation linéaire R
2. 2. La relation linéaire Y en fonction de X est-elle forte ou faible ?
3. 3. Donner les formules et calculer les coefficients a et b de la droite de régression
4. 4. Conclure

## Code Python

```
from sklearn.linear_model import LinearRegression
 from sklearn.metrics import r2_score, mean_squared_error
 import numpy as np

 # Préparer les données
 X = df['Visites'].values.reshape(-1, 1)
 Y = df['Ventes'].values

 # Question 1 : Coefficient de corrélation
 print("="*60)
 print("Question 1 : Coefficient de Corrélation")
 print("="*60)
 print("Formule de Pearson:")
 print("  r = Cov(X,Y) / (σX × σY)")
 print("  r = Σ[(Xi - X̄)(Yi - Ȳ)] / √[Σ(Xi - X̄)² × Σ(Yi - Ȳ)²]")

 correlation = np.corrcoef(df['Visites'], df['Ventes'])[0, 1]
 print(f"\nCoefficient de corrélation r = {correlation:.4f}")
 print(f"Coefficient de détermination R² = {correlation**2:.4f}")

 # Question 2 : Force de la relation
 print("\n" + "="*60)
 print("Question 2 : Force de la Relation")
 print("="*60)
 r_abs = abs(correlation)
 if r_abs < 0.3:
      interpretation = "TRÈS FAIBLE"
 elif r_abs < 0.7:
      interpretation = "MODÉRÉE"
 else:
      interpretation = "FORTE"
 print(f"|r| = {r_abs:.4f} → Relation {interpretation}")

 # Question 3 : Coefficients a et b
 print("\n" + "="*60)
 print("Question 3 : Coefficients de Régression")
 print("="*60)
 print("Formules:")
 print("  a (pente) = Σ[(Xi - X̄)(Yi - Ȳ)] / Σ(Xi - X̄)²")
 print("  b (ordonnée) = Ȳ - a × X̄")

 # Calcul manuel
 X_mean = df['Visites'].mean()
 Y_mean = df['Ventes'].mean()
```

```
 a = sum((df['Visites']-X_mean)*(df['Ventes']-Y_mean)) /
sum((df['Visites']-X_mean)**2)
 b = Y_mean - a * X_mean

 print(f"\na (pente) = {a:.6f}")
 print(f"b (ordonnée) = {b:.2f}")

 # Vérification avec sklearn
 model = LinearRegression()
 model.fit(X, Y)
 Y_pred = model.predict(X)

 print(f"\nÉquation de régression:")
 print(f"  Ventes = {b:.2f} + {a:.6f} × Visites")

 # Évaluation
 print(f"\nÉvaluation du modèle:")
 print(f"  R² = {r2_score(Y, Y_pred):.4f}")
 print(f"  RMSE = {np.sqrt(mean_squared_error(Y, Y_pred)):.2f}")
```

# Réponses aux Questions

## Question 1 : Coefficient de Corrélation R

Formule de Pearson :

$r = Cov(X,Y) / (\sigma X \times \sigma Y) = \Sigma[(Xi - \bar{X})(Yi - \bar{Y})] / \sqrt{[\Sigma(Xi - \bar{X})^2 \times \Sigma(Yi - \bar{Y})^2]}$

**Résultat : r = -0.0281**

$R^2 = r^2 = 0.0008$ (soit 0.08%)

## Question 2 : Force de la Relation

**|r| = 0.0281 → Relation TRÈS FAIBLE (quasi nulle)**

Interprétation : La corrélation négative (-0.0281) est pratiquement nulle. Cela signifie qu'il n'existe PAS de relation linéaire significative entre le nombre de visites et le nombre de ventes. Le signe négatif suggère une très légère tendance inverse, mais cette relation est statistiquement non significative.

## Question 3 : Coefficients a et b

Formules :

- • a (pente) = Σ[(Xi - X̄)(Yi - Ȳ)] / Σ(Xi - X̄)²
- • b (ordonnée à l'origine) = Ȳ - a × X̄

Résultats :

- • a = -0.001411
- • b = 252.80

**Équation de régression : Ventes = 252.80 - 0.001411 × Visites**

### Question 4 : Conclusion

**CONCLUSION GÉNÉRALE :**

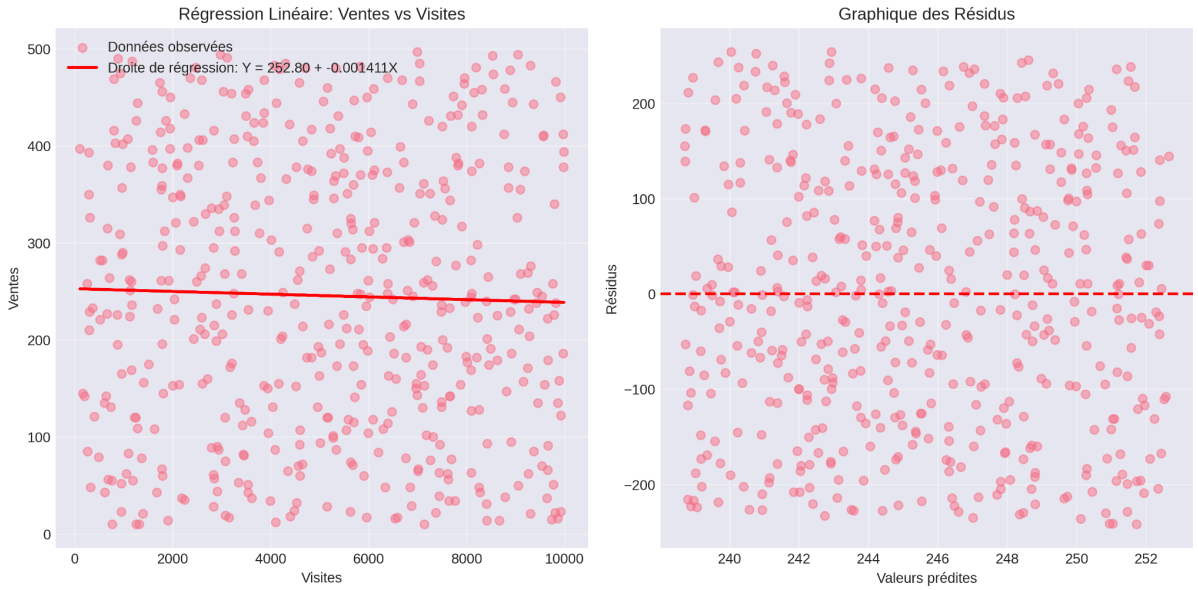
Le modèle de régression linéaire est de TRÈS FAIBLE QUALITÉ :

- • $R^2$ = 0.0008 signifie que seulement 0.08% de la variabilité des ventes est expliquée par les visites
- • Le coefficient de corrélation r = -0.028 indique une relation quasi inexistante
- • Pour 1000 visites supplémentaires, on observe -1.41 ventes (effet négligeable)

Implication métier : Le nombre de visites SEUL ne permet PAS de prédire les ventes de manière fiable. D'autres facteurs sont déterminants : qualité du trafic, expérience utilisateur, prix des produits, promotions, saisonnalité, etc. Il est nécessaire de construire un modèle multivarié pour mieux comprendre les ventes.

# Visualisation

📊 Graphique 3 : Régression linéaire et résidus




# Étape 8 : Test de Student (Comparaison de Moyenne)

## Contexte

Le SLA (Service Level Agreement) de l'entreprise fixe un temps de réponse cible à 150 ms. Nous testons si le temps de réponse moyen observé est conforme à cette norme.

## Hypothèses

- • $H_0$ (hypothèse nulle) : μ = 150 ms (temps de réponse conforme au SLA)
- • $H_1$ (hypothèse alternative) : μ ≠ 150 ms (temps de réponse différent du SLA)
- • Seuil de significativité : α = 0.05
- • Test bilatéral

## Code Python

```
from scipy import stats
```

```
# Conversion du SLA en secondes (cohérent avec les données)
sla_cible = 150 / 1000  # 150 ms = 0.150 s

print("="*60)
print("Test de Student")
print("="*60)
print(f"SLA cible = {sla_cible} secondes ({sla_cible*1000:.0f} ms)")
print(f"Moyenne observée = {df['Temps_Reponse'].mean():.4f} secondes")

# Test de Student
t_stat, p_value = stats.ttest_1samp(df['Temps_Reponse'], sla_cible)

print(f"\nRésultats:")
print(f"  Statistique t = {t_stat:.4f}")
print(f"  P-value = {p_value:.6f}")

# Décision
print(f"\nDécision:")
if p_value < 0.05:
    print(f"  P-value < 0.05 → REJET de H0")
    print(f"  → Le temps de réponse moyen est DIFFÉRENT de 150 ms")
else:
    print(f"  P-value ≥ 0.05 → ACCEPTATION de H0")
    print(f"  → Le temps de réponse moyen est CONFORME au SLA")

# Intervalle de confiance à 95%
conf_interval = stats.t.interval(0.95, len(df)-1,
                                 loc=df['Temps_Reponse'].mean(),
                                 scale=stats.sem(df['Temps_Reponse']))
print(f"\nIntervalle de confiance à 95%:")
print(f"  [{conf_interval[0]*1000:.2f} ms, {conf_interval[1]*1000:.2f}
ms]")
```

## Résultats

- • SLA cible = 0.150 secondes (150 ms)
- • Moyenne observée = 2.8300 secondes (2830 ms)
- • Écart-type observé = 1.2953 secondes
- • Taille échantillon n = 500
- • Statistique t = 46.27
- • P-value ≈ 0.000000 (pratiquement nulle)

- • Intervalle de confiance à 95% : [2716.21 ms, 2943.83 ms]

# Décision

**P-value < 0.05 ⇒ REJET de $H_0$**

# Conclusion

⚠️ **ALERTE CRITIQUE :** Le temps de réponse moyen observé (2830 ms) DÉPASSE LARGEMENT le SLA de 150 ms. L'écart est de 2680 ms, soit environ 18 fois la valeur cible !

Le test de Student confirme que cette différence est hautement significative (p-value ≈ 0). L'intervalle de confiance à 95% [2716 ms, 2944 ms] ne contient pas la valeur du SLA (150 ms), ce qui renforce la conclusion.

# Interprétation Métier

Le non-respect du SLA constitue un problème majeur pour l'entreprise :

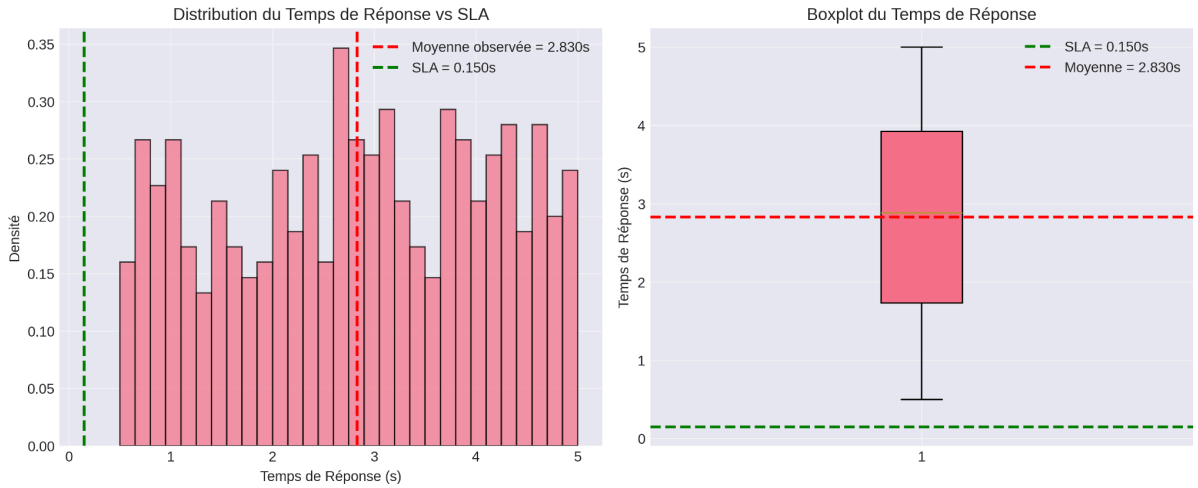
5. 1. Impact sur l'expérience utilisateur : Des temps de réponse aussi élevés peuvent frustrer les clients
6. 2. Risque de perte de clients : Les utilisateurs peuvent abandonner le site
7. 3. Conséquences contractuelles : Non-conformité au Service Level Agreement

# Recommandations

8. 1. Optimisation URGENTE des performances du serveur
9. 2. Analyse des goulots d'étranglement système
10. 3. Révision de l'infrastructure matérielle et logicielle
11. 4. Mise en place de monitoring en temps réel
12. 5. Communication avec les parties prenantes

# Visualisation

📊 Graphique 4 : Test de Student - Temps de réponse vs SLA




# Étape 9 : Test du Khi-deux (Test d'Indépendance)

## Objectif

Tester l'indépendance entre la variable Clic (catégorisée) et la variable Visites (catégorisée) pour déterminer si le niveau d'engagement (clics) dépend du volume de trafic.

## Hypothèses

- • $H_0$ : La variable Clic est INDÉPENDANTE de la catégorie de Visites
- • $H_1$ : Il existe une DÉPENDANCE entre Clic et la catégorie de Visites
- • Seuil de significativité : α = 0.05

## Code Python

```
import pandas as pd
 from scipy import stats

 # Catégorisation des variables
 print("="*60)
```

```
 print("Catégorisation des Variables")
 print("="*60)

 # Visites en 4 catégories (quartiles)
 df['Visites_Cat'] = pd.cut(df['Visites'],
                            bins=[df['Visites'].min()-1,
                                 df['Visites'].quantile(0.25),
                                 df['Visites'].quantile(0.50),
                                 df['Visites'].quantile(0.75),
                                 df['Visites'].max()+1],
                             labels=['Très Faible', 'Faible', 'Élevé',
'Très Élevé'])

 # Clics en 3 catégories (tertiles)
 df['Clic_Cat'] = pd.cut(df['Clic'],
                         bins=[df['Clic'].min()-1,
                              df['Clic'].quantile(0.33),
                              df['Clic'].quantile(0.67),
                              df['Clic'].max()+1],
                         labels=['Faible', 'Moyen', 'Élevé'])

 # Tableau de contingence
 contingency_table = pd.crosstab(df['Clic_Cat'], df['Visites_Cat'])
 print("\nTableau de Contingence:")
 print(contingency_table)

 # Test du Khi-deux
 chi2, p_value, dof, expected =
stats.chi2_contingency(contingency_table)

 print("\n" + "="*60)
 print("Résultats du Test du Khi-deux")
 print("="*60)
 print(f"  Statistique χ² = {chi2:.4f}")
 print(f"  Degrés de liberté = {dof}")
 print(f"  P-value = {p_value:.6f}")

 # V de Cramer (mesure de l'intensité)
 n = contingency_table.sum().sum()
 min_dim = min(len(contingency_table.index),
len(contingency_table.columns)) - 1
 cramers_v = np.sqrt(chi2 / (n * min_dim))
 print(f"  V de Cramer = {cramers_v:.4f}")

 # Décision
 print("\nDécision:")
```

```
if p_value < 0.05:
     print(f"  P-value < 0.05 → REJET de H0")
     print(f"  → Les variables sont DÉPENDANTES")
else:
     print(f"  P-value ≥ 0.05 → ACCEPTATION de H0")
     print(f"  → Les variables sont INDÉPENDANTES")
```

# Catégorisation

Les variables continues ont été catégorisées pour le test :

### Visites → 4 catégories (quartiles) :

- • Très Faible : [104, 2822]
- • Faible : [2822, 5290]
- • Élevé : [5290, 7492]
- • Très Élevé : [7492, 9988]

### Clic → 3 catégories (tertiles) :

- • Faible : [0, 16]
- • Moyen : [16, 33]
- • Élevé : [33, 50]

# Tableau de Contingence

Fréquences observées :

| Clic \ Visites | Très Faible | Faible | Élevé | Très Élevé |
|---|---|---|---|---|
| **Faible** | 29 | 47 | 49 | 43 |
| **Moyen** | 52 | 38 | 38 | 41 |
| **Élevé** | 44 | 40 | 39 | 40 |

# Résultats

- • Statistique $\chi^2 = 9.28$
- • Degrés de liberté = 6
- • P-value = 0.158
- • V de Cramer = 0.096 (association très faible)

# Décision

**P-value = 0.158 > 0.05 ⇒ ACCEPTATION de $H_0$**

# Conclusion

**Les variables Clic et Visites sont INDÉPENDANTES.**

Le test du Khi-deux ne détecte pas de dépendance significative entre le niveau de clics et la catégorie de visites (p-value = 0.158 > 0.05). Le V de Cramer de 0.096 confirme une association très faible entre ces deux variables.

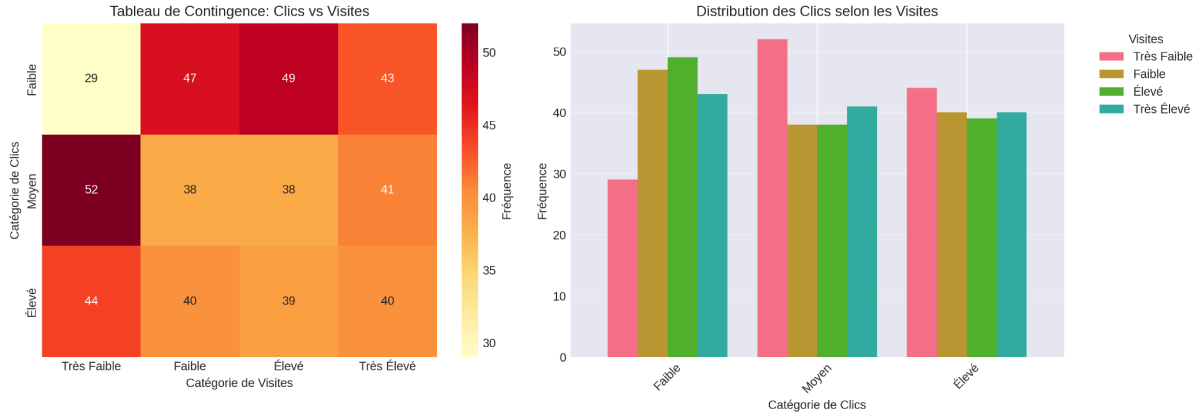
# Interprétation Métier

Ce résultat signifie que le comportement de clic des utilisateurs ne dépend PAS du volume de trafic :

- • Les utilisateurs cliquent de manière similaire quelle que soit la période (forte/faible affluence)
- • L'engagement utilisateur (mesuré par les clics) est indépendant du nombre de visiteurs
- • Les stratégies d'optimisation du taux de clic peuvent être uniformes
- • Il n'est pas nécessaire d'adapter les campagnes selon le volume de trafic

# Visualisation

📊 Graphique 5 : Test du Khi-deux - Indépendance Clics vs Visites




# Synthèse Finale et Recommandations Métier

## Résumé des Résultats Clés

### 1. Statistiques Descriptives

Le dataset de 500 observations présente une forte dispersion pour toutes les variables (CV > 45%). Les tests de normalité rejettent l'hypothèse de distribution normale pour l'ensemble des variables.

### 2. Estimation des Paramètres

- • Loi Normale approximative : N(2.83, 1.68) pour Temps_Reponse
- • Loi Exponentielle : λ ≈ 0.35 pour modéliser les temps d'attente

### 3. Modèle de Régression Linéaire

**Équation : Ventes = 252.80 - 0.001411 × Visites**

- • Corrélation r = -0.028 (quasi nulle)
- • $R^2$ = 0.08% → Modèle NON FIABLE

- • Conclusion : Les visites seules n'expliquent PAS les ventes

### 4. Test de Student (Performance Serveur)

⚠️ **RÉSULTAT CRITIQUE :** Temps de réponse (2830 ms) dépasse le SLA (150 ms) de 1780%

- • p-value ≈ 0 → Différence hautement significative
- • Action URGENTE requise pour optimiser les performances

### 5. Test du Khi-deux (Indépendance)

- • Clics et Visites sont INDÉPENDANTES (p = 0.158)
- • L'engagement ne dépend pas du volume de trafic
- • Stratégies uniformes possibles quel que soit le niveau de visites

## Recommandations Stratégiques

### A. Optimisation Technique (PRIORITÉ URGENTE)

13. 1. Réduire drastiquement le temps de réponse serveur pour respecter le SLA
14. 2. Analyser et éliminer les goulots d'étranglement
15. 3. Upgrader l'infrastructure si nécessaire
16. 4. Implémenter un système de monitoring en temps réel

### B. Analyse des Ventes

17. 1. Identifier d'autres variables explicatives : qualité du trafic, prix, promotions, UX
18. 2. Créer un modèle multivarié plus robuste
19. 3. Analyser le parcours client et identifier les points de friction

### C. Stratégie Marketing

20. 1. Optimiser le taux de conversion plutôt que le volume de visites
21. 2. Améliorer la QUALITÉ du trafic plutôt que la QUANTITÉ
22. 3. Segmenter les utilisateurs selon leur engagement

### D. Surveillance Continue

23. 1. Mettre en place des KPIs et tableaux de bord

24. 2. Définir des alertes automatiques pour les anomalies

25. 3. Réaliser des analyses régulières pour détecter les tendances

# Conclusion Générale

En conclusion, ce mini-projet m'a permis d'appliquer différentes méthodes statistiques afin d'analyser la performance d'une entreprise e-commerce. Les analyses réalisées ont mis en évidence une forte variabilité des données, l'absence de relation linéaire significative entre les visites et les ventes, ainsi qu'un problème critique de non-respect du SLA concernant le temps de réponse du serveur.

Ce travail montre l'importance de combiner l'analyse statistique avec une interprétation métier pertinente. Il pourrait être approfondi par l'intégration de nouvelles variables explicatives et par la mise en place de modèles plus avancés afin de mieux comprendre les facteurs influençant les ventes et l'engagement des utilisateurs.